AI/ML PORTFOLIO PROJECT

Intelligent Helpdesk Ticket Classificati



Flask web app that classifies IT support tickets with TF-IDF and Logistic Regress
guidance

NLP | Machine Learning | IT Service Management

Thangaraj T | tmthangaraj1980@gmail.com | +91 9952045978
github.com/tmthangaraj1980-AIML

Live demo URL from README could not be verified from the build environment

Executive overview

- Employee types a support ticket in plain English
- Model predicts a category with a confidence score and top 3 alternatives
- App shows assigned team, priority, ETA and troubleshooting steps
- Flask backend, Bootstrap 5 front end using the Fetch API
- Trained on 1,000 labelled tickets in the notebook

16

categories in the trained model

94%

accuracy on 200 held-out tickets

1,564

TF-IDF features

1 Ticket | 2 Clean | 3 TF-IDF | 4 Classify | 5 Lookup | 6 Display

Problem statement

Slow triage

Agents read and tag every ticket by hand.

Mis-routing

Wrong team means reassignment and delay.

Inconsistent priority

Urgent items such as phishing can sit in the queue.

Repeat answers

The same fix steps are typed again and again.

1 Ticket | 2 Clean | 3 TF-IDF | 4 Classify | 5 Lookup | 6 Display

Objectives and use cases

Objectives

- Classify ticket text automatically
- Show confidence and top 3
- Route to the right team
- Give first-line fix steps

Users

- Employees
- IT service desk
- Team leads

Benefits

- Faster routing
- Consistent guidance
- Less manual tagging

1 Ticket | 2 Clean | 3 TF-IDF | 4 Classify | 5 Lookup | 6 Display

Technology stack

Machine learning

- scikit-learn: TF-IDF + Logistic Regression
- Joblib model persistence
- Pandas, NumPy, NLTK stopwords (notebook)

Backend

- Flask 3.0.3, gunicorn
- JSON /predict endpoint
- Python 3.12

Frontend

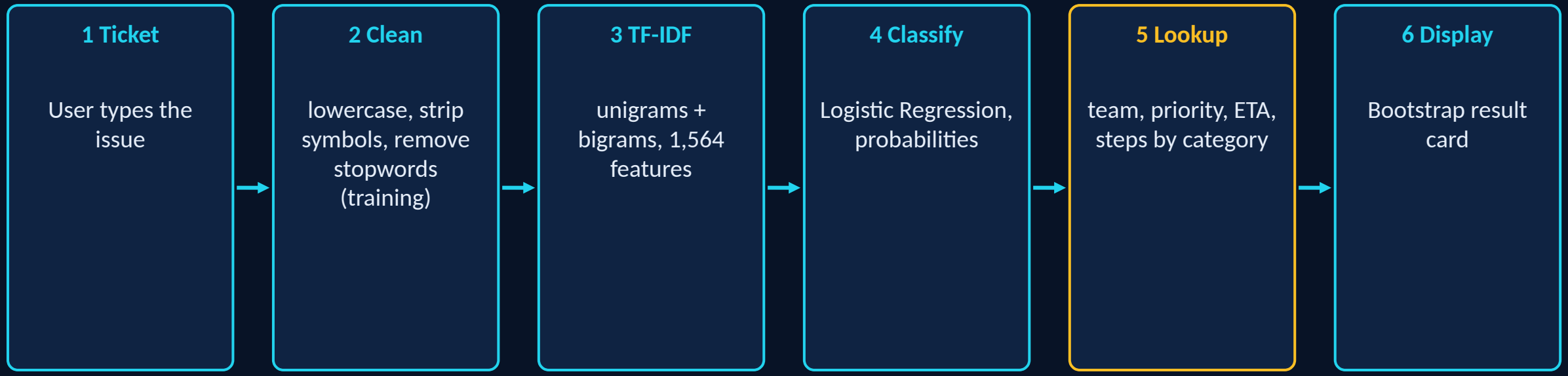
- HTML5, CSS3, Bootstrap 5
- JavaScript Fetch API
- Sample tickets, copy solution

Deployment

- Render URL documented in README
- Not verifiable from build environment

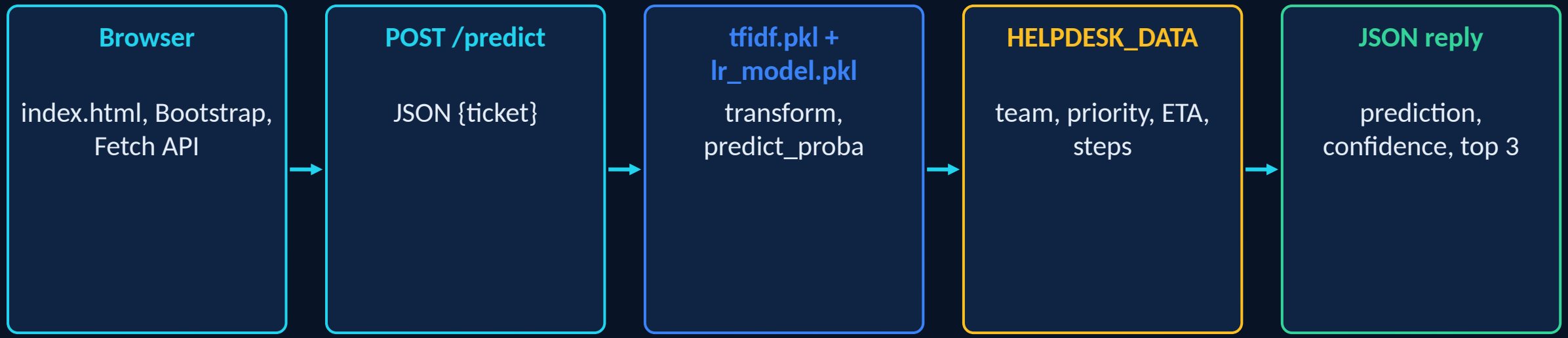
1 Ticket | 2 Clean | 3 TF-IDF | 4 Classify | 5 Lookup | 6 Display

End-to-end project flow



Note: the live /predict route does not apply the cleaning step; see the limitations slide.

System architecture



Error handling

Empty ticket returns HTTP 400. Exceptions return HTTP 500 JSON. 404 and 500 pages are referenced in code but the templates are not in the repository.

1 Ticket | 2 Clean | 3 TF-IDF | 4 Classify | 5 Lookup | 6 Display

Dataset

Item	Value (notebook)
Rows / columns	1,000 / 6 (ticket, category, ticket_id, priority, dept, timestamp)
Categories	16, roughly 57 to 73 tickets each
Split	800 train / 200 test, stratified
Cleaning	lowercase, letters only, NLTK stopwords removed
Observation	short, templated sentences; looks synthetic (not confirmed)

The 16 categories include six not in the 10-category knowledge base: Login Issue, Payroll, Attendance, HR Query, Leave Management, Unclassified.

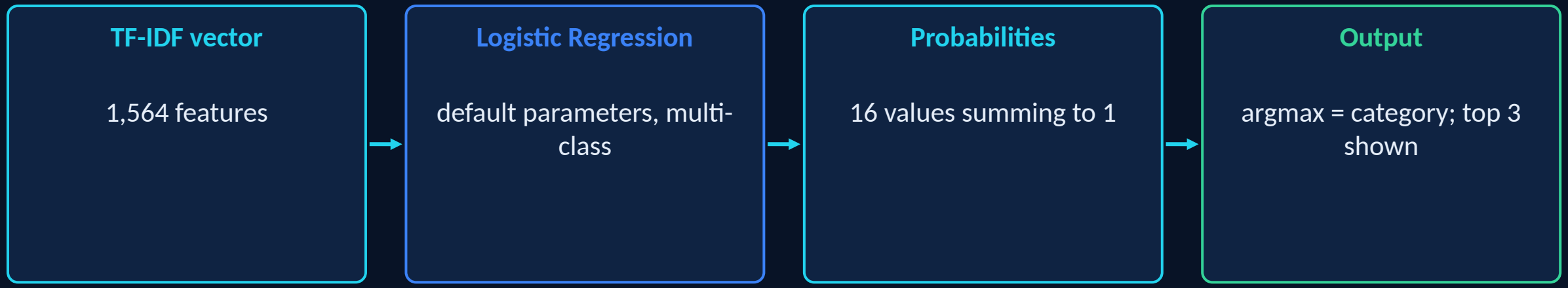
Text preprocessing and TF-IDF



Fit on train only

The vectorizer is fitted on the 800 training tickets and only transforms the test set, which avoids vocabulary leakage.

Model: Logistic Regression



Why this model

A linear model on TF-IDF is fast, needs little data, and its probabilities are easy to explain. It is a strong baseline before trying transformers.

Which outputs are ML and which are rules

Output	Source
Category	Machine learning (Logistic Regression)
Confidence and top 3	Machine learning (predict_proba)
Assigned team	Rule: dictionary by category
Priority	Rule: fixed per category
Estimated resolution time	Rule: fixed per category
Troubleshooting steps	Rule: 5 hand-written steps per category

Knowledge-base gap

- Dictionary covers 10 categories
- Model predicts 16
- The other 6 get a generic reply: IT Support, Medium, 1 Business Day

Results and evaluation (verified from the notebook)

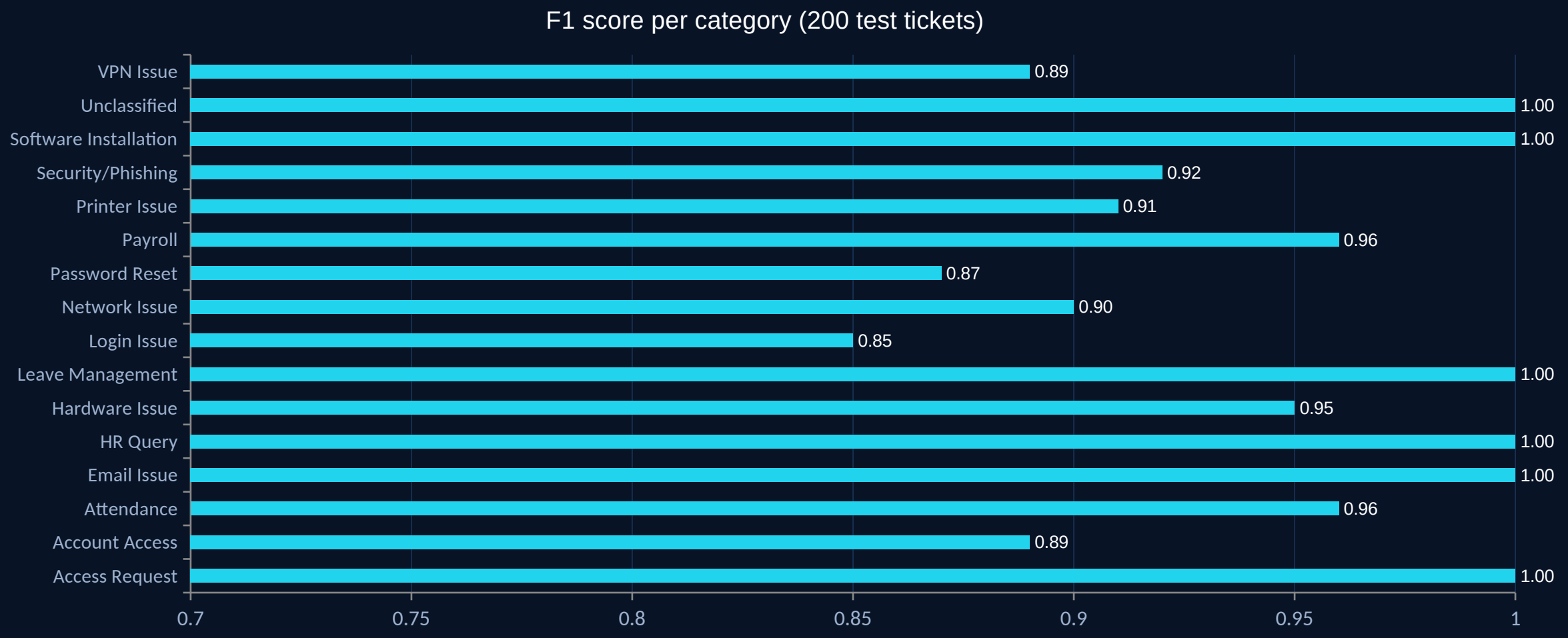
94%	0.94	0.94	0.94
accuracy	weighted precision	weighted recall	weighted F1

How to read this

- Macro average precision 0.95, recall 0.94, F1 0.94
- Evaluated once on 200 tickets; no cross-validation
- High scores on short templated text may overstate real-world performance

1 Ticket | 2 Clean | 3 TF-IDF | 4 Classify | 5 Lookup | 6 Display

Per-class F1 score



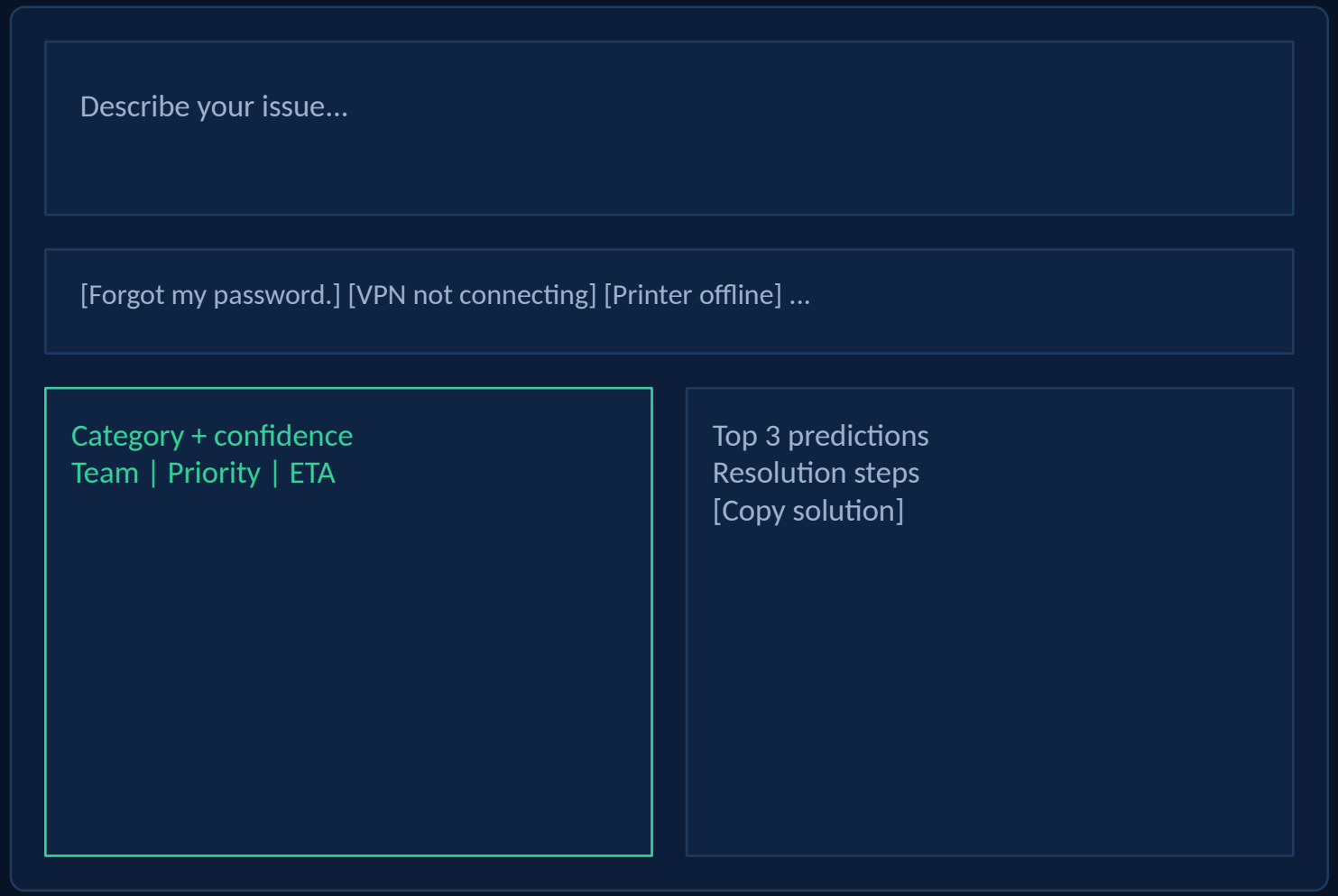
1 Ticket | 2 Clean | 3 TF-IDF | 4 Classify | 5 Lookup | 6 Display

Sample classifications (saved model run)

Ticket	Top 3 predicted categories (probability)
My VPN is not connecting from home	VPN Issue 52.6%, Access Request 5.4%, Password Reset 4.6%
I received a suspicious email asking for my password	Security/Phishing 43.6%, Password Reset 13.8%, Email Issue 5.3%
The printer is out of toner	Printer Issue 89.2%, Network Issue 0.9%, Hardware Issue 0.8%

Confidence is a probability spread over 16 classes, so a correct answer can show roughly 50%.

Application interface (schematic from index.html)



1. Ticket input and sample tickets
2. ML prediction + confidence
3. Rule-based team, priority, ETA
4. Steps and copy button

Illustrative schematic, not a real screenshot

1 Ticket | 2 Clean | 3 TF-IDF | 4 Classify | 5 Lookup | **6 Display**

API contract

Item	Detail
Request	POST /predict with JSON {"ticket": "..."}
Success	prediction, confidence, assigned_team, priority, eta, resolution[], top_predictions[]
Empty ticket	HTTP 400 with an error message
Failure	HTTP 500 with the exception text

Technical challenges and limitations

Train / serve mismatch

- Notebook cleans text before training
- Live /predict passes raw text
- My check on the held-out split: about 1 point lower (approximate stopwords)

Knowledge base

- 6 of 16 categories have no entry
- Generic fallback reply
- Priority is fixed, not learned

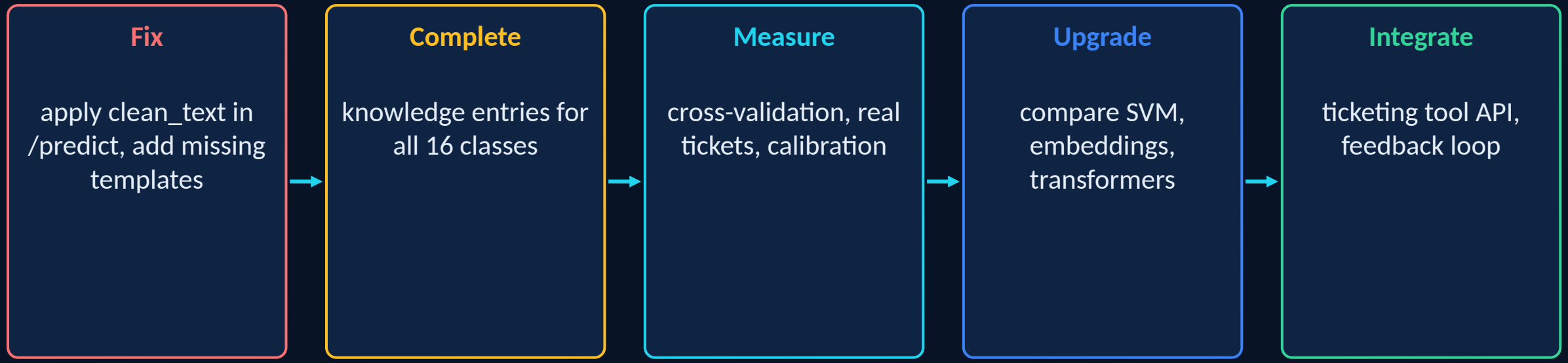
Evidence

- One split, no cross-validation
- Dataset looks templated
- Confidence is not calibrated

Packaging

- 404 and 500 templates missing
- Notebook text says Streamlit, app is Flask
- Run uses debug=True locally

Improvements and roadmap



1 Ticket | 2 Clean | 3 TF-IDF | 4 Classify | 5 Lookup | 6 Display

Interview preparation

1. Is everything shown a model prediction?

No. Only category and probabilities. Team, priority, ETA and steps come from a dictionary keyed by category.

2. How good is the model?

94% accuracy and 0.94 weighted F1 on 200 held-out tickets from one stratified split; I would confirm with cross-validation on real tickets.

3. Why TF-IDF with Logistic Regression?

Fast, explainable baseline for short text with probabilities; transformers are the next step if data grows.

4. What bug would you fix first?

Training cleans text but /predict does not; I would apply the same clean_text function at inference.

5. Why can confidence look low on correct answers?

Probability is spread over 16 classes and is not calibrated; calibration or a threshold review would help.

Resources and missing details

Available

- Repository: code, notebook, models, dataset
- README live URL (below)
- Notebook metrics (accuracy 0.94)

github.com/tmthangaraj1980-AIML/AI_Helpdesk_Ticket_Classifier

Missing or unverified

- Real screenshots
- Verified live demo (could not reach the URL from the build environment)
- Cross-validation results
- Real ticket data

Conclusion

Skills demonstrated

- Text classification pipeline
- TF-IDF + Logistic Regression
- Flask JSON API and Bootstrap UI
- Separating ML output from rules
- Critical code review

Project impact

- Faster, consistent ticket routing
- Clear improvement path
- Honest evaluation story

Thangaraj T | tmthangaraj1980@gmail.com | +91 9952045978